

Candidate Data Transformer: Technical Design

Problem

Recruiting data arrives fragmented across disparate sources with conflicting formats and missing fields. This system consolidates these sources into a canonical merged profile using deterministic identity resolution and explainable merge policies. The result is a clean, validated, and highly traceable candidate dataset.

Architecture & Pipeline

Pipeline: Sources → Parsers → Canonical Schema → Normalizers → Blocking → Similarity → Match Decision → Merge Engine → Confidence/Provenance → Validation → Projection → Output JSON/UI.

Canonical Schema

A shared Pydantic model (`CanonicalCandidate`) used across all layers. It encapsulates identity, professional experience, background, traceability, and merge metadata. By standardizing the contract early, downstream layers remain source-agnostic.

Merge Policy

- Merges only deterministic `MATCH` groups; avoids inventing data.
- Prefers non-empty, highest-confidence sources, and exact cross-source agreement.
- Nested lists (skills, experience) are merged using field-by-field unioning.

Confidence & Provenance

- **Confidence:** Rule-based calculation combining base source confidence, cross-source agreement, and field completeness.
- **Provenance:** Every merged field records the selected value, contributing sources, source IDs, confidence score, and the specific merge reason. Fully transparent.

Runtime Configuration

Business rules live outside the code in JSON configs: output shaping, matching thresholds, source confidences, and alias maps. This enables rapid tuning without deployment overhead.

Edge Cases Handled

- Conflicting names/titles are resolved via confidence weighting.
- Common name collisions are mitigated by combining multiple blocking keys (e.g., email + phone + last name).
- `POSSIBLE_MATCH` cases are flagged for review rather than incorrectly merged.

Complexity

Instead of $O(n^2)$ all-pairs matching, blocking reduces comparisons to $O(p)$ where $p \ll n^2$. The system easily scales vertically by tuning blocking keys.

Future Improvements

- Integrate a human-in-the-loop UI for reviewing `POSSIBLE_MATCH` candidates.
- Persist blocking indexes for incremental, stateful matching at larger scale.